```
const CACHE_NAME = 'look-v3';

const APP_SHELL = [
  './index.html',
  './manifest.json',
  './icon.svg'
];

const CDN_ASSETS = [
  'https://cdn.jsdelivr.net/npm/tesseract.js@4/dist/tesseract.min.js',
  'https://cdn.jsdelivr.net/npm/xlsx@0.18.5/dist/xlsx.full.min.js'
];

const PRECACHE = APP_SHELL.concat(CDN_ASSETS);

self.addEventListener('install', event => {
  event.waitUntil((async () => {
    const cache = await caches.open(CACHE_NAME);
    await cache.addAll(PRECACHE);
    await self.skipWaiting();
  })());
});

self.addEventListener('activate', event => {
  event.waitUntil((async () => {
    const names = await caches.keys();
    await Promise.all(names.filter(name => name !== CACHE_NAME).map(name => cach
es.delete(name)));
    await self.clients.claim();
  })());
});

self.addEventListener('fetch', event => {
  const request = event.request;
  if (request.method !== 'GET') return;

  const url = new URL(request.url);
  const appShellUrls = APP_SHELL.map(path => new URL(path, self.location.href).h
ref);
  const isAppShell = appShellUrls.includes(url.href) || (request.mode === 'navig
ate' && url.origin === self.location.origin);
  const isCdn = url.hostname === 'cdn.jsdelivr.net';

  if (isAppShell) {
    event.respondWith(cacheFirst(request, request.mode === 'navigate' ? './index
.html' : null));
    return;
  }

  if (isCdn) {
    event.respondWith(cdnCacheFirst(request, event));
    return;
  }

  event.respondWith(networkFirst(request));
});

async function cacheFirst(request, fallbackPath) {
  const cache = await caches.open(CACHE_NAME);
  const matchRequest = fallbackPath || request;
  const cached = await cache.match(matchRequest);
  if (cached) return cached;
```

```
  const response = await fetch(request);
  if (response && response.ok) await cache.put(matchRequest, response.clone());
  return response;
}

async function cdnCacheFirst(request, event) {
  const cache = await caches.open(CACHE_NAME);
  const cached = await cache.match(request);
  const update = (async () => {
    const response = await fetch(request);
    if (response && response.ok) await cache.put(request, response.clone());
    return response;
  })();
  if (cached) {
    event.waitUntil(update);
    return cached;
  }
  return await update;
}

async function networkFirst(request) {
  const cache = await caches.open(CACHE_NAME);
  try {
    const response = await fetch(request);
    if (response && response.ok) await cache.put(request, response.clone());
    return response;
  } catch (error) {
    const cached = await cache.match(request);
    if (cached) return cached;
    throw error;
  }
}
```